Milestone_two

Daichi Larocque

```
library(bcdata)
```

```
Attaching package: 'bcdata'

The following object is masked from 'package:stats':

    filter
```

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.1.6
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.1     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.1

-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks bcdata::filter(), stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to beco
```

```
library(tinytable)
```

```
#bc_sei_data_raw <- bcdc_get_data("b-c-socioeconomic-index")
```

I uused [@tinytable] to code this

```
#write_csv(bc_sei_data_raw, "bc_sei_data_raw.csv")
```

After writing the dataset onto my computer, I have made a copy of the dataset to keep the original file.

```
socioeconomic_index_raw <- read_csv("bc_sei_data_raw.csv")
```

```
Rows: 319 Columns: 11
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (1): MUN_NAME_2021
dbl (10): CSD_UID, CALENDAR_YEAR, AVERAGE_AGE, POPULATION_ESTIMATE, TOTAL_IN...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Cleaning

Let's get a sense of the data here.

```
head(socioeconomic_index_raw)
```

```
# A tibble: 6 x 11
  CSD_UID MUN_NAME_2021   CALENDAR_YEAR AVERAGE_AGE POPULATION_ESTIMATE
    <dbl> <chr>                   <dbl>       <dbl>               <dbl>
1 5901003 Elkford                  2023        37.3                2949
2 5901006 Sparwood                 2023        38.2                4452
3 5901012 Fernie                   2023        38.6                6778
4 5901017 East Kootenay A          2023        45.8                2003
5 5901019 East Kootenay B          2023        45.8                2148
6 5901022 Cranbrook                2023        43.5               22057
# i 6 more variables: TOTAL_INDEX_0_100 <dbl>, SEI_INDEX_0_100 <dbl>,
#   ECON_0_100 <dbl>, EDUC_0_100 <dbl>, HEALTH_0_100 <dbl>,
#   COMMUNITY_0_100 <dbl>
```

First, I will get rid of columns that I do not need.

```
cleaned_socioeconomic_index <- socioeconomic_index_raw |>
  select(-CALENDAR_YEAR)
```

Now, I will only include the municipalities of interest.

```
cleaned_socioeconomic_index <- cleaned_socioeconomic_index |>
  filter(
    MUN_NAME_2021 %in% c(
    "Anmore", "Belcarra", "Bowen Island", "Burnaby",
    "Coquitlam", "Delta", "Langley", "Lions Bay", "Maple Ridge",
    "Metro Vancouver A", "New Westminster", "North Vancouver",
    "Pitt Meadows", "Port Coquitlam", "Port Moody", "Richmond",
    "Surrey", "Vancouver", "West Vancouver", "White Rock", "Central Saanich",
    "Colwood", "Esquimalt", "Highlands", "Juan de Fuca (Part 1)",
    "Juan de Fuca (Part 2)", "Langford", "Metchosin", "North Saanich",
    "Oak Bay", "Saanich", "Sidney", "Sooke", "Victoria", "View Royal"
  )
)
```

In the dataset, some municipalities are named twice because they contain sub-regions or districts. It is important that we give these better identifiers so that we can be more precise in our analysis.

```
cleaned_socioeconomic_index <- cleaned_socioeconomic_index |>
  mutate(
    MUN_NAME_2021 = case_when(
    CSD_UID == "5915002" ~ "City of Langley",
    CSD_UID == "5915001" ~ "Township of Langley",
    CSD_UID == "5915020" ~ "Electoral Area A (UBC)",
    CSD_UID == "5915051" ~ "City of North Vancouver",
    CSD_UID == "5915046" ~ "District of North Vancouver",
    CSD_UID == "5917054" ~ "Juan de Fuca (Part 1, Shirley, Jordan River, Malahat)",
    CSD_UID == "5917056" ~ "Juan de Fuca (Part 2, Port Renfrew)",
    TRUE ~ MUN_NAME_2021
  )
)
```

Let's also rename the columns

```r
cleaned_socioeconomic_index <- cleaned_socioeconomic_index |>
  rename(
    Municipality = MUN_NAME_2021,
    Average_Age = AVERAGE_AGE,
    Population_2023 = POPULATION_ESTIMATE,
    Total_Index = TOTAL_INDEX_0_100,
    Socioeconomic_Index = SEI_INDEX_0_100,
    Economics = ECON_0_100,
    Education = EDUC_0_100,
    Health = HEALTH_0_100,
    Community = COMMUNITY_0_100
  )
```

Now, lets make it so that the index data only goes up to two decminal points.

```r
index_decimal <- 2

cleaned_socioeconomic_index <- cleaned_socioeconomic_index |>
  mutate(
    across(
      c(
        Total_Index,
        Socioeconomic_Index,
        Economics,
        Education,
        Health,
        Community
    ),
    round, digits = index_decimal
  )
)
```

```
Warning: There was 1 warning in `mutate()`.
i In argument: `across(...)`.
Caused by warning:
! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
Supply arguments directly to `.fns` through an anonymous function instead.

  # Previously
  across(a:b, mean, na.rm = TRUE)

  # Now
```

```
  across(a:b, \(x) mean(x, na.rm = TRUE))
```

```
cleaned_socioeconomic_index |>
  as_tibble()
```

```
# A tibble: 37 x 10
   CSD_UID Municipality           Average_Age Population_2023 Total_Index
     <dbl> <chr>                        <dbl>           <dbl>       <dbl>
 1 5915001 Township of Langley           39.7          154103        56.1
 2 5915002 City of Langley               41.4           33626        46.8
 3 5915004 Surrey                        38.4          653040        52.0
 4 5915007 White Rock                    52.1           24294        51.0
 5 5915011 Delta                         41.8          120048        54.1
 6 5915015 Richmond                      42.1          235082        54.9
 7 5915020 Electoral Area A (UBC)        37.8           21150        60.0
 8 5915022 Vancouver                     40.9          732593        55.8
 9 5915025 Burnaby                       40.7          283588        56.0
10 5915029 New Westminster               40.7           88410        54.2
# i 27 more rows
# i 5 more variables: Socioeconomic_Index <dbl>, Economics <dbl>,
#   Education <dbl>, Health <dbl>, Community <dbl>
```

```
age_decimal <- 0

cleaned_socioeconomic_index <- cleaned_socioeconomic_index |>
  mutate(
    across(
      c(
        Average_Age
      ),
      round, digits = age_decimal
  )
)

cleaned_socioeconomic_index |>
  as_tibble()
```

```
# A tibble: 37 x 10
   CSD_UID Municipality           Average_Age Population_2023 Total_Index
     <dbl> <chr>                        <dbl>           <dbl>       <dbl>
 1 5915001 Township of Langley             40          154103        56.1
```

```
 2 5915002 City of Langley                41           33626        46.8
 3 5915004 Surrey                         38          653040        52.0
 4 5915007 White Rock                     52           24294        51.0
 5 5915011 Delta                          42          120048        54.1
 6 5915015 Richmond                       42          235082        54.9
 7 5915020 Electoral Area A (UBC)         38           21150        60.0
 8 5915022 Vancouver                      41          732593        55.8
 9 5915025 Burnaby                        41          283588        56.0
10 5915029 New Westminster                41           88410        54.2
# i 27 more rows
# i 5 more variables: Socioeconomic_Index <dbl>, Economics <dbl>,
#   Education <dbl>, Health <dbl>, Community <dbl>
```

Tests

I have two basic tests I want to run before diving into more contextual ones. The first is to make sure all variables are proper class. I am doing this by using the stopifnot function, which will stop running the code if it detects a variable that is not the proper class. The second is to make sure there are no negative values in the numeric variables. This code uses a logical if/else function to print out true or false statements depending on whether number less than zero are found in each variable.

```
stopifnot(
  "Municipality must be a character" = is.character(cleaned_socioeconomic_index$Municipality)
  "Population must be numeric" = is.numeric(cleaned_socioeconomic_index$Population_2023),
  "Average age must be numeric" = is.numeric(cleaned_socioeconomic_index$Average_Age),
  "Total index must be numeric" = is.numeric(cleaned_socioeconomic_index$Total_Index),
  "SEI Index muste be numeric" = is.numeric(cleaned_socioeconomic_index$Socioeconomic_Index)
  "Community must be numeric" = is.numeric(cleaned_socioeconomic_index$Community),
  "Health must be numeric" = is.numeric(cleaned_socioeconomic_index$Health),
  "Education must be numeric" = is.numeric(cleaned_socioeconomic_index$Education),
  "Economics must be numeric" = is.numeric(cleaned_socioeconomic_index$Economics)
)
```

```
numeric_variables <- c(
  "Population_2023", "Average_Age", "Total_Index", "Socioeconomic_Index",
  "Community", "Health", "Education", "Economics"
)

for(
  SEI_var in numeric_variables) {
```

```
    if(any(cleaned_socioeconomic_index[[SEI_var]] < 0, na.rm = TRUE)) {
      stop(
        paste("Stop, there are negative values in the", SEI_var, "variable"))
 } else {
      message(
        paste("Passed, there are no negative values in the", SEI_var, "variable")
  )
 }
}
```

```
Passed, there are no negative values in the Population_2023 variable
```

```
Passed, there are no negative values in the Average_Age variable
```

```
Passed, there are no negative values in the Total_Index variable
```

```
Passed, there are no negative values in the Socioeconomic_Index variable
```

```
Passed, there are no negative values in the Community variable
```

```
Passed, there are no negative values in the Health variable
```

```
Passed, there are no negative values in the Education variable
```

```
Passed, there are no negative values in the Economics variable
```

Looks like all the tests passed and there are no negative numbers where there should not be.